

LECTURE 30

File Systems: Concepts and Structure

CSC323 · Operating Systems

COMSATS University Islamabad, Lahore Campus

In This Lecture

- The file concept: what a file abstracts away, and the attributes every file carries
- File operations, and specifically why `open()` and `close()` exist as separate bookkeeping steps
- Access methods: sequential access vs. direct (random) access
- Directory structures: single-level, two-level, and tree-structured, and what problem each one solves that the previous one didn't
- The layered file-system implementation, from application code down to the physical device

The File Concept

A file is the operating system's basic unit of logical storage: a named collection of related information, recorded on secondary storage.

- The file is a deliberate abstraction — to the application programmer, a file is just a name and a stream (or a set) of bytes
- The messy physical reality underneath — which specific disk blocks hold the data, whether they are contiguous or scattered, which blocks are still free — is hidden entirely behind the file system
- A program that wants to store a document never has to think in terms of sectors, tracks, or spare-block pools; it thinks in terms of a file name and the bytes it writes to or reads from that name

File Attributes

Every file carries a set of attributes, maintained by the file system rather than by the file's own contents.

Attribute	What it records
Name	The human-readable name the file is identified by within its directory
Identifier (inode number)	A unique number the file system itself uses internally to identify the file, independent of its name
Type	What kind of file this is (where the file system or OS distinguishes types at all)
Size	The file's current size, in bytes
Protection	Who is allowed to read, write, or execute the file (the full subject of Lecture 31)
Timestamps	When the file was created, last modified, and last accessed

A file's name and its identity are not the same thing

The name is for humans; the identifier (often an inode number) is what the file system actually uses to locate the file's data. This is why a file can be renamed without disturbing anything that has it open, and why two directory entries (hard links) can point at the very same file.

File Operations

The operating system provides a small set of system calls to manipulate files.

- Create — allocate space for a new file and add an entry for it to a directory
- Write — append (or overwrite) data at the current position within the file
- Read — retrieve data starting at the current position within the file
- Reposition within a file (seek) — move the file's current position without transferring any data
- Delete — remove the file's directory entry and release the space it occupied
- Truncate — erase a file's contents while keeping all of its attributes, resetting its length to zero

Why Bother with `open()` and `close()` at All?

- Every operation above needs the file system to first search the relevant directory structure for the file's entry, confirm the caller is permitted to access it, and set up in-memory bookkeeping before anything can actually happen
- Doing all of that again on every single `read()` or `write()` call would be enormously wasteful, especially for a program issuing thousands of small reads to the same file

`open()` and `close()` are pure bookkeeping, not data transfer

`open()` does the search-and-setup work exactly once, creates an entry in an in-memory open-file table, and hands the calling program back a cheap handle (a file descriptor) that every subsequent read or write can use directly, with no directory search required. `close()` simply tears that bookkeeping down again once the program is done, freeing the open-file-table entry for reuse.

Access Methods

A file system also has to decide how a program is allowed to move through a file's data — and most systems support at least two distinct models.

Sequential access

Read (or write) records in order, one after another, exactly like reading a tape — the file's own traditional model

Direct (random) access

Jump straight to any block of the file by its block number, with no need to pass through the blocks before it

Sequential vs. Direct Access, in Practice

- Sequential access is the older, simpler model: each read picks up exactly where the previous one left off, and each write appends to the end — the same discipline a magnetic tape drive physically requires
- It works well for files naturally processed start-to-finish, like a log file or a video stream
- Direct access lets a program request block N of a file directly, without reading blocks 0 through N-1 first — essential for database-style usage, where an engine locates one specific record by its known block number rather than scanning every record before it

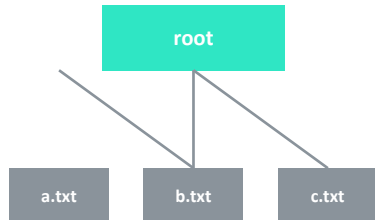
Sequential access is still possible on a direct-access file

Supporting direct access does not take sequential access away — a file system built for direct access can always be read sequentially simply by requesting blocks 0, 1, 2, ... in order. The reverse is not true: a strictly sequential-access medium cannot efficiently jump to an arbitrary block, which is exactly why direct access is the harder capability to provide and the one that matters for database-style workloads.

Directory Structure

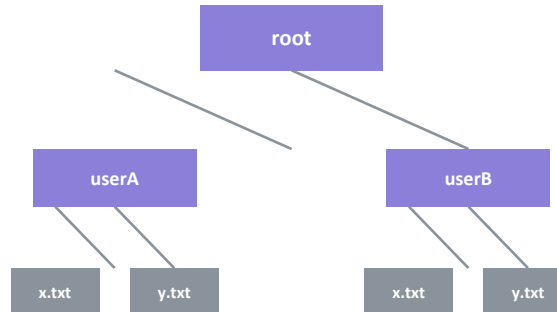
A file system needs a directory structure to organize many files in a way that keeps their names distinct and their organization sensible. This structure evolved in stages, each solving a problem the previous stage left unsolved.

Single-Level



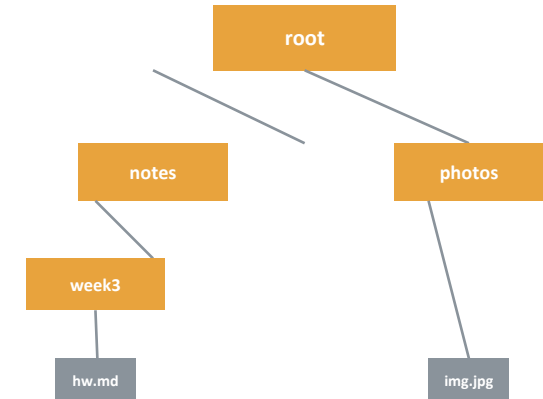
One directory for the entire disk — every file needs a globally unique name

Two-Level



One directory per user — solves naming conflicts between users, not within one user's files

Tree-Structured



Directories nest to any depth — files named by a full path; the structure almost every file system uses today

Single-Level and Two-Level Directories

- Single-level: the simplest possible scheme — one directory holds every file on the entire system, for every user
- Trivial to implement, but does not scale past a single user: every file name must be unique across the entire disk, so two users each wanting notes.txt means one of them loses
- There is also no way to group related files together; everything lives in one flat, undifferentiated list
- Two-level: the next step gives each user their own directory, with one master directory mapping user names to their individual directories
- This fixes the cross-user naming conflict — two users can each have their own notes.txt, since a file's full identity is now (user, filename), not just filename
- What it does not fix: organization within one user's own files — a user with hundreds of files is still stuck with one flat list

Tree-Structured Directory

- The structure virtually every file system uses today: directories can contain other directories, to any depth, forming a tree
- A file (or directory) is identified by a path — the sequence of directory names leading from some starting point down to it
- Every process maintains a current directory, which lets a relative path (notes/week3.txt) stand in for the full absolute path (/home/amina/notes/week3.txt)
- This solves both earlier problems at once: two files can share a name in different directories anywhere in the tree, and a user can organize files into as many nested groupings as they like

Tree-structured directories still need a deletion policy

Deleting a directory that still contains files (or sub-directories) raises a choice every tree-structured file system has to make explicitly: refuse the deletion until the directory is empty, or recursively delete everything beneath it. Getting this wrong — silently deleting an entire subtree a user expected to be protected — causes real, unrecoverable data loss.

File-System Structure

Application Programs

Issue system calls like `open()`, `read()`, `write()` on file names



Logical File System

Manages directory structure, file metadata, and protection; translates a file name into its identifier



File-Organization Module

Translates logical block numbers (within a file) into physical block numbers (on the device)



Basic File System

Issues generic commands to the appropriate device driver to read/write physical blocks



I/O Control

Device drivers and interrupt handlers that actually move data between main memory and the device controller



Devices

The physical disk, SSD, or other storage hardware

From a read() Call to Real Disk I/O

Lecture 3 introduced the system call as the formal boundary between an application and the kernel — a `read()` call is exactly where that boundary gets exercised in the file-system context.

1. The application calls `read()` on a file handle it already obtained from `open()`
2. The logical file system looks up the open-file-table entry the handle refers to, confirms the caller's permissions, and determines which logical blocks of the file satisfy the request
3. The file-organization module translates those logical block numbers into the actual physical block numbers the data lives at on the device
4. The basic file system issues a generic command (“read physical block P”) down to the appropriate device driver, without caring what file, or even what file system, that block belongs to
5. I/O control — the device driver and its interrupt handler — carries out the actual transfer between the device controller and main memory, and signals completion back up the stack

Key Takeaways

- A file is the OS's basic logical unit of storage — a named collection of data that hides the physical scattering of disk blocks behind a small set of attributes (name, identifier, type, size, protection, timestamps)
- File operations include create, write, read, reposition, delete, and truncate; open and close are separate bookkeeping operations that set up and tear down an in-memory open-file-table entry exactly once
- Sequential access reads a file in order, like a tape; direct (random) access jumps straight to any block by number — essential for database-style workloads
- Directory structures evolved from single-level (global naming conflicts), to two-level (conflicts solved between users), to tree-structured (arbitrary nesting, path-based naming) — the structure virtually every modern file system uses
- A file system is implemented as layers — application programs, logical file system, file-organization module, basic file system, I/O control, and devices — and a single `read()` system call passes down through every one of them on its way to an actual disk access